Embedding

Abstract

Purpose

- build a text embedder that can capture characteristics of texts specified by user instructions.

- instead of generic text similarity, we want the embedding to change based on what the user wants.

Methodology

- texts with similar semantic meaning have similar answers following the instruction → similar embeddings

Results

- follows instructions better than RoBERTa and LLAMA (2-7b)
- qualitative analysis of clustering outcomes shows high interpretability

---

Summary

Depending on the use-case, what information is deemed important from a text corpus changes. Prompts given by users serve as instructions on how to perceive the given text. This instruction following capabilities are not captured well by existing text embedders. Existing embedders are static meaning they make one generalized vector which may not be adequate for all scenarios.

This problem is solved by the proposed framework in this paper - InBedders or Instruction-following Embedder framework.

The paper achieves the following tasks:

- addressing the lack of instruction following capabilities by existing text embedders
- Introducing a suite of tasks to evaluate the instruction following capabilities
- an approach to extract information form embedding clusters

The intuitive approach is to embed the instruction along with the input context.

instruction tuning is one possible way to tuning, but their embeddings are weak for retrieval. There is no guarantee that they generalize well to new instructions.

In the paper a better approach is proposed:

- feed the instruction along with the input to a language model
- the language model is fine tuned to give short responses
- the responses along with the context is given to the embedder

The first step is to preprocess the data. The language models often reply with an initial paragraph ('based on the given input') that is meant to the make the chatbot seem more interactive. It, however does not contribute to the instruction following capabilities.

Training is done using 11 abstractive QA datasets (200k paragraph-question-answer triplets).

Three model architectures have been considered. RoBerta-large, OPT models, and LLama 2.

Roberta is an encoder only bidirectionally masked language model. i.e it learns by predicting missing words in a sentence allowing. masked loss is used in this case

OPT models are autoregressive. [markovian]

LLama is an instruction following chat model.

with these answers, the instruction following embeddings are generated. however, Inbedder doesnt necessarily use final output text. It extracts hidden state and applies different aggregation methods.

Avg-Gen (Average of Generated Tokens)	Captures output meaning

Avg-PPT (Average of Prompt Tokens)	Captures input meaning

1st-Gen (First Generated Token)	Captures immediate model response

Last-Gen (Last Generated Token)	Captures final model output state

Avg-All (All Tokens)	Captures full sequence meaning

These embeddings are evaluated on three tasks

- Semantic Textual Similarity

  a custom made version of the origin STS dataset is used for this task. the similarity levels are also scaled down between 0 and 1. The metric is spearman correlation of cosine similarity between pairs of embeddings and labeled data

- Instruction Robustness

  given a triplet of different instructions. similar intent embeddings should be closer to each other than those which aren't. Two success rate is defined to capture the same idea for intent and emotion. (similar intent closer)(similar emotion closer)

- goal based clustering

  For each clustering task, 10 correct instructions are generated by instructing GPT-4 to paraphrase the original task instructions. Similarly, 10 implicit instructions and 10 incorrect instructions are produced by GPT-4

  the difference in performance between (correct,incorrect) and (inplicit,incorrect) instruction pairs are used

---

Q1

What is the motivation for this work

Improving text embeddings so that they can adapt to different user needs. Traditional embedding models fail to represent text dynamically based on context. Search, Retrieval and clustering tasks are hindered by this bottleneck on existing embedders.

why don't people have a trivial solution?

a straightforward solution is to embed the concatenated instruction and input. Generic embeddings represent the texts in a form that can be used in textual similarity tasks, but are not very apt for instruction following

previous solutions, why are they inadequate?

- sentence-T5
- sentence Bert
- simCSE - Simple Contrastive learning of sentence embeddings

lack the ability to address user-specific objectives.

Traditional embedding models only generate one fixed embedding per text, which represents the general semantic meaning of the text.

They do not account for different ways a user might be interested for the same text

Q2

What is the proposed solution (hypothesis, idea, design)?

A novel framework, InBEDDER, has been proposed to solve the said problem.

- fine tunes language model on a dataset of 200,00 paragraph-question-answer triplets
- when an instruction is given by a user, it is treated as a question and an answer is generated using the fine tuned LM
- the answer is then embedded

How does it represent an improvement?

There is significant improvement in instruction following capabilities based on the proposed scheme of evaluation, in comparison to LLMs (llama-2-7b) and encoder based LMs (roberta-large)

Q3

What is the evaluation of the solution?

the instruction following capabilities are evaluated based on 3 schemes

- selection of closer sentence to the anchor sentence based on two different instructions

  it builds upon the STS-B by introducing IntenEmotion

  STS-B dataset has pairwise,
- an instruction following sentence similarity task
- clustering the same corpus under various instructions

What would you need to know before you would adopt this approach?

- how to fine tune the LM’s
- what are the compute requirements (they mentioned using A100s)

Q4

What are the paper’s contributions

(author’s opinion)

- instruction following text embedding framework
- comprehensive assessment
- Approach to extract explanations from embeddings clusters

(My opinion - bird’s eye)

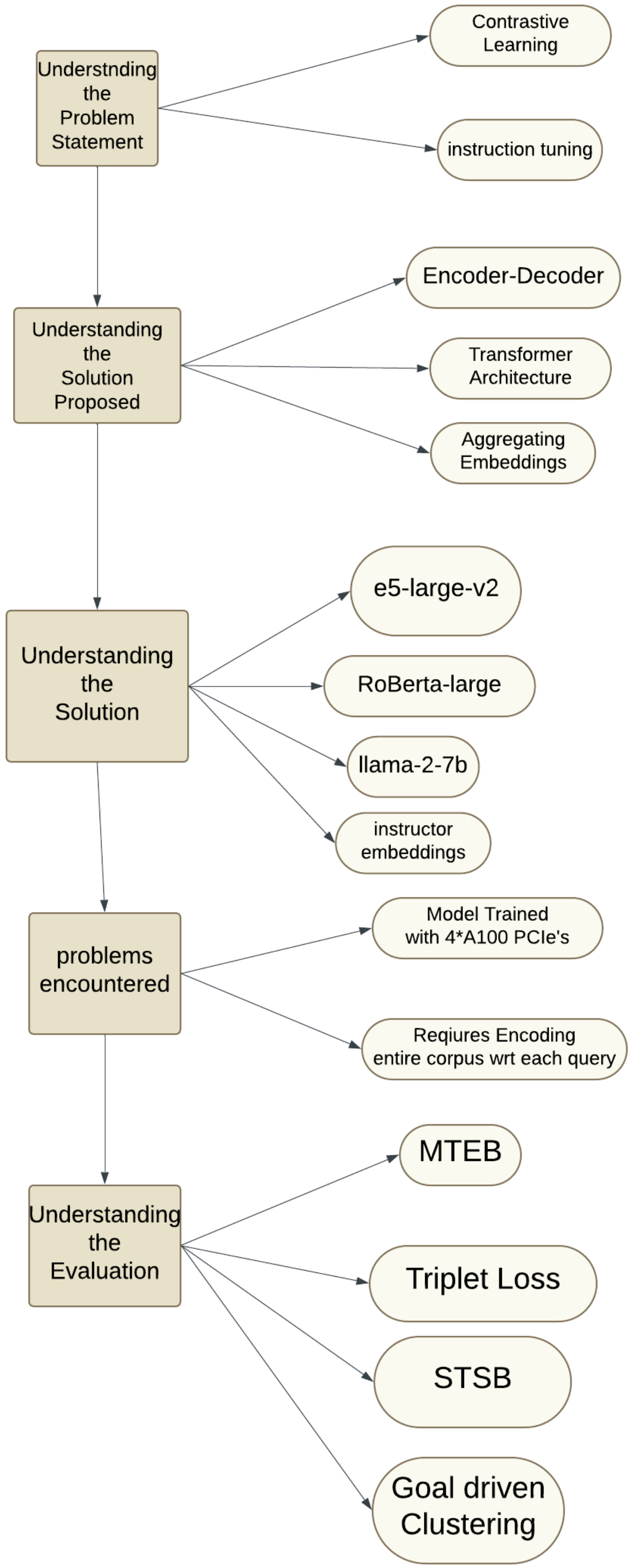
Gives rise to a myriad of applications in search, retrieval and recommender system space

- content summarization
- customer support

Q5

What are the future directions for this research

Paper Prototype


Understnding the Problem Statement
Contrastive Learning
instruction tuning
Understanding the Solution Proposed
Encoder-Decoder
Transformer Architecture
Aggregating Embeddings
Understanding the Solution
e5-large-v2
RoBerta-large
llama-2-7b
instructor embeddings
problems encountered
Model Trained with 4*A100 PCIe's
Reqiures Encoding entire corpus wrt each query
Understanding the Evaluation
MTEB
Triplet Loss
STSB
Goal driven Clustering

Useful Links

Understanding the problem statement

- contrastive learning

  https://www.v7labs.com/blog/contrastive-learning-guide

- instruction tuning

  https://aclanthology.org/2023.findings-acl.71.mp4

understanding the solution proposed

- Encoder-Decoder

  https://medium.com/analytics-vidhya/encoders-decoders-sequence-to-sequence-architecture-5644efbb3392

- Transformer Architecture

  https://jalammar.github.io/illustrated-transformer/https://www.youtube.com/watch?v=wjZofJX0v4M

- aggregating embeddings

  https://blog.ml6.eu/the-art-of-pooling-embeddings-c56575114cf8

understanding the solution

- e5 large v2

  https://dataloop.ai/library/model/intfloat_e5-large-v2/

- bert

  https://jalammar.github.io/illustrated-bert/

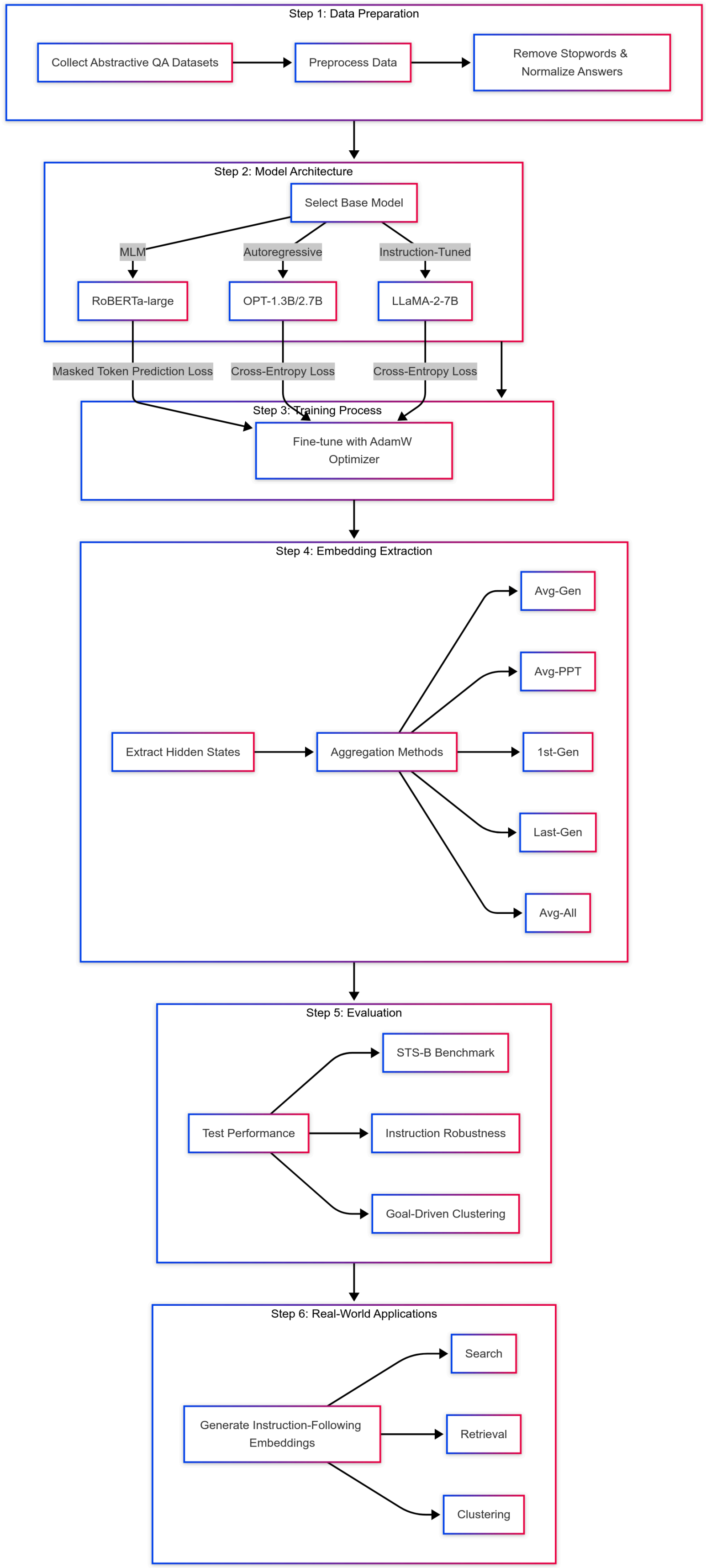
- Roberta large

  https://towardsdatascience.com/roberta-1ef07226c8d8/

- llama-2-7b
- instructor embeddings

  https://aclanthology.org/2023.emnlp-main.657.mp4
- opt models

Implementation


Step 1: Data Preparation
Collect Abstractive QA Datasets
Preprocess Data
Remove Stopwords & Normalize Answers
Step 2: Model Architecture
Select Base Model
MLM
Autoregressive
Instruction-Tuned
RoBERTa-large
OPT-1.3B/2.7B
LLaMA-2-7B
Masked Token Prediction Loss
Cross-Entropy Loss
Cross-Entropy Loss
Step 3: Training Process
Fine-tune with AdamW Optimizer
Step 4: Embedding Extraction
Extract Hidden States
Aggregation Methods
Avg-Gen
Avg-PPT
1st-Gen
Last-Gen
Avg-All
Step 5: Evaluation
Test Performance
STS-B Benchmark
Instruction Robustness
Goal-Driven Clustering
Step 6: Real-World Applications
Generate Instruction-Following Embeddings
Search
Retrieval
Clustering